GitHub Project Setup Guide

Initial Repository Configuration

1. Repository Creation

```bash
# Create new repository on GitHub
# Name: recruitment-matching-platform
# Description: AI-powered intelligent recruitment matching system
# Visibility: Private (initially) or Public

# Clone and initialize locally
git clone git@github.com:your-org/recruitment-matching-platform.git
cd recruitment-matching-platform

# Set up initial structure
mkdir -p backend frontend ml infrastructure scripts docs .github/workflows
```

2. Branch Protection Rules

For `main` branch:

- ✅ Require pull request reviews before merging (2 reviewers)
- ✅ Dismiss stale pull request approvals when new commits are pushed
- ✅ Require status checks to pass before merging
- ✅ Require branches to be up to date before merging
- ✅ Require conversation resolution before merging
- ✅ Require signed commits (recommended)
- ✅ Include administrators in restrictions
- ✅ Restrict who can push to matching branches
- ✅ Allow force pushes: disabled
- ✅ Allow deletions: disabled

For `develop` branch:

- ✅ Require pull request reviews before merging (1 reviewer)
- ✅ Require status checks to pass before merging
- ✅ Allow force pushes: disabled

3. GitHub Actions Secrets

Add these secrets in repository settings:

```
# Docker Registry
DOCKER_USERNAME=your_username
DOCKER_PASSWORD=your_password

# AWS (if using)
AWS_ACCESS_KEY_ID=your_key
AWS_SECRET_ACCESS_KEY=your_secret
AWS_REGION=us-east-1

# Database Credentials
DB_PASSWORD=your_db_password
MONGODB_PASSWORD=your_mongo_password

# API Keys
OPENAI_API_KEY=your_openai_key (if using)
HUGGINGFACE_TOKEN=your_hf_token

# Application Secrets
JWT_SECRET_KEY=your_jwt_secret
ENCRYPTION_KEY=your_encryption_key

# Monitoring
SLACK_WEBHOOK_URL=your_slack_webhook (for notifications)
```

GitHub Issue Templates

Bug Report Template

Create `.github/ISSUE_TEMPLATE/bug_report.md`:

```markdown
```

```
---
name: Bug Report
about: Create a report to help us improve
title: '[BUG] '
labels: bug
assignees: ''
---

**Describe the bug**
A clear and concise description of what the bug is.

**To Reproduce**
Steps to reproduce the behavior:
1. Go to '...'
2. Click on '....'
3. Scroll down to '....'
4. See error

**Expected behavior**
A clear and concise description of what you expected to happen.

**Screenshots**
If applicable, add screenshots to help explain your problem.

**Environment:**
 - OS: [e.g. Ubuntu 22.04]
 - Browser [e.g. chrome, safari]
 - Version [e.g. 22]
 - Component [backend/frontend/ml]

**Additional context**
Add any other context about the problem here.

**Logs**
```

Paste relevant logs here

```
**Possible Solution**
If you have suggestions on how to fix this, please describe.
```

Feature Request Template

Create `.github/ISSUE_TEMPLATE/feature_request.md`:

```markdown
```

```
---
name: Feature Request
about: Suggest an idea for this project
title: '[FEATURE] '
labels: enhancement
assignees: ''
---

**Is your feature request related to a problem? Please describe.**
A clear and concise description of what the problem is. Ex. I'm always frustrated wh

**Describe the solution you'd like**
A clear and concise description of what you want to happen.

**Describe alternatives you've considered**
A clear and concise description of any alternative solutions or features you've cons

**Component**
Which component does this feature affect?
- [ ] Backend API
- [ ] Frontend UI
- [ ] ML Models
- [ ] Infrastructure
- [ ] Documentation

**Priority**
- [ ] Critical
- [ ] High
- [ ] Medium
- [ ] Low

**Additional context**
Add any other context or screenshots about the feature request here.

**Acceptance Criteria**
- [ ] Criterion 1
- [ ] Criterion 2
- [ ] Criterion 3
```

Pull Request Template

Create `.github/PULL_REQUEST_TEMPLATE.md`:

```markdown
```

Description
<!-- Provide a detailed description of your changes -->

Type of Change
- [ ] Bug fix (non-breaking change which fixes an issue)
- [ ] New feature (non-breaking change which adds functionality)
- [ ] Breaking change (fix or feature that would cause existing functionality to not
- [ ] Documentation update
- [ ] Refactoring (no functional changes)
- [ ] Performance improvement
- [ ] Test coverage improvement

Related Issue
<!-- Link to the issue this PR addresses -->
Closes #(issue number)

Changes Made
<!-- List the main changes in bullet points -->
-
-
-

Testing
<!-- Describe the tests you ran and how to reproduce them -->

Test Coverage
- [ ] Unit tests added/updated
- [ ] Integration tests added/updated
- [ ] E2E tests added/updated
- [ ] Manual testing performed

Test Evidence
```bash
# Paste test results here
```

Screenshots (if applicable)
<!-- Add screenshots to help explain your changes -->

Checklist
- [ ] My code follows the project's style guidelines
- [ ] I have performed a self-review of my own code
- [ ] I have commented my code, particularly in hard-to-understand areas
- [ ] I have made corresponding changes to the documentation
- [ ] My changes generate no new warnings
- [ ] I have added tests that prove my fix is effective or that my feature works

```
- [ ] New and existing unit tests pass locally with my changes
- [ ] Any dependent changes have been merged and published

## Performance Impact
<!-- Describe any performance implications -->
- [ ] No performance impact
- [ ] Performance improved
- [ ] Performance degraded (explain why it's acceptable)

## Security Considerations
<!-- Describe any security implications -->
- [ ] No security impact
- [ ] Security improved
- [ ] New security measures added

## Deployment Notes
<!-- Any special deployment steps required? -->
- [ ] Database migrations needed
- [ ] Environment variables added/changed
- [ ] Configuration changes required
- [ ] Dependencies added/updated

## Reviewer Guidance
<!-- Any specific areas you want reviewers to focus on? -->
```

CI/CD Workflows

Backend CI Workflow

Create `.github/workflows/backend-ci.yml`:

```yaml
```

```yaml
name: Backend CI

on:
  pull_request:
    branches: [develop, main]
    paths:
      - 'backend/**'
      - '.github/workflows/backend-ci.yml'
  push:
    branches: [develop, main]
    paths:
      - 'backend/**'

jobs:
  lint-and-test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:14
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: test_db
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432

      redis:
        image: redis:7
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 6379:6379

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
```

```yaml
      with:
        python-version: '3.10'
        cache: 'pip'

    - name: Install dependencies
      run: |
        cd backend
        pip install -r requirements.txt
        pip install -r requirements-dev.txt

    - name: Lint with flake8
      run: |
        cd backend
        flake8 app/ --count --select=E9,F63,F7,F82 --show-source --statistics
        flake8 app/ --count --exit-zero --max-complexity=10 --max-line-length=127

    - name: Format check with Black
      run: |
        cd backend
        black --check app/

    - name: Type check with mypy
      run: |
        cd backend
        mypy app/

    - name: Run tests with pytest
      env:
        DATABASE_URL: postgresql://postgres:postgres@localhost:5432/test_db
        REDIS_URL: redis://localhost:6379
      run: |
        cd backend
        pytest tests/ -v --cov=app --cov-report=xml --cov-report=html

    - name: Upload coverage to Codecov
      uses: codecov/codecov-action@v3
      with:
        file: ./backend/coverage.xml
        flags: backend
        name: backend-coverage

    - name: Security check with bandit
      run: |
        cd backend
        bandit -r app/ -f json -o bandit-report.json

    - name: Upload test results
```

```yaml
        if: always()
        uses: actions/upload-artifact@v3
        with:
          name: test-results
          path: backend/htmlcov/

  docker-build:
    runs-on: ubuntu-latest
    needs: lint-and-test

    steps:
      - uses: actions/checkout@v3

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v2

      - name: Build Docker image
        uses: docker/build-push-action@v4
        with:
          context: ./backend
          push: false
          tags: recruitment-backend:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

Frontend CI Workflow

Create `.github/workflows/frontend-ci.yml`:

```yaml
```

```yaml
name: Frontend CI

on:
  pull_request:
    branches: [develop, main]
    paths:
      - 'frontend/**'
      - '.github/workflows/frontend-ci.yml'
  push:
    branches: [develop, main]
    paths:
      - 'frontend/**'

jobs:
  lint-and-test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'
          cache: 'npm'
          cache-dependency-path: frontend/package-lock.json

      - name: Install dependencies
        run: |
          cd frontend
          npm ci

      - name: Lint with ESLint
        run: |
          cd frontend
          npm run lint

      - name: Format check with Prettier
        run: |
          cd frontend
          npm run format:check

      - name: Type check with TypeScript
        run: |
          cd frontend
          npm run type-check
```

```yaml
    - name: Run unit tests
      run: |
        cd frontend
        npm run test:coverage

    - name: Upload coverage to Codecov
      uses: codecov/codecov-action@v3
      with:
        file: ./frontend/coverage/coverage-final.json
        flags: frontend
        name: frontend-coverage

    - name: Build application
      run: |
        cd frontend
        npm run build

    - name: Upload build artifacts
      uses: actions/upload-artifact@v3
      with:
        name: frontend-build
        path: frontend/build/

  e2e-tests:
    runs-on: ubuntu-latest
    needs: lint-and-test

    steps:
      - uses: actions/checkout@v3

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install dependencies
        run: |
          cd frontend
          npm ci
          npx playwright install --with-deps

      - name: Run E2E tests
        run: |
          cd frontend
          npm run test:e2e
```

```yaml
      - name: Upload test results
        if: always()
        uses: actions/upload-artifact@v3
        with:
          name: playwright-report
          path: frontend/playwright-report/
```

ML Model CI Workflow

Create `.github/workflows/ml-ci.yml`:

```yaml
```

```yaml
name: ML Pipeline CI

on:
  pull_request:
    branches: [develop, main]
    paths:
      - 'ml/**'
      - '.github/workflows/ml-ci.yml'
  push:
    branches: [develop, main]
    paths:
      - 'ml/**'

jobs:
  test-ml-pipeline:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'
          cache: 'pip'

      - name: Install dependencies
        run: |
          cd ml
          pip install -r requirements.txt
          pip install -r requirements-dev.txt

      - name: Lint with flake8
        run: |
          cd ml
          flake8 src/ --count --max-line-length=127 --statistics

      - name: Format check with Black
        run: |
          cd ml
          black --check src/

      - name: Run data validation tests
        run: |
          cd ml
          pytest tests/test_data_validation.py -v
```

```yaml
      - name: Run feature engineering tests
        run: |
          cd ml
          pytest tests/test_features.py -v

      - name: Run model tests
        run: |
          cd ml
          pytest tests/test_models.py -v --cov=src --cov-report=xml

      - name: Model performance check
        run: |
          cd ml
          python scripts/evaluate_models.py --threshold 0.80

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          file: ./ml/coverage.xml
          flags: ml
          name: ml-coverage
```

Deployment Workflow

Create `.github/workflows/deploy.yml`:

```yaml
```

```yaml
name: Deploy to Production

on:
  push:
    branches:
      - main
    tags:
      - 'v*.*.*'

jobs:
  deploy-backend:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Login to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}

      - name: Build and push Backend
        uses: docker/build-push-action@v4
        with:
          context: ./backend
          push: true
          tags: |
            your-org/recruitment-backend:latest
            your-org/recruitment-backend:${{ github.sha }}

      - name: Deploy to production
        run: |
          # Add deployment script here
          # kubectl apply -f infrastructure/kubernetes/backend/
          echo "Deploying backend to production..."

  deploy-frontend:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
```

```yaml
          node-version: '18'

      - name: Build frontend
        run: |
          cd frontend
          npm ci
          npm run build

      - name: Deploy to S3 / CDN
        run: |
          # Add deployment to S3 or static hosting
          echo "Deploying frontend..."

      - name: Invalidate CDN cache
        run: |
          # Add CDN cache invalidation
          echo "Invalidating CDN cache..."

  notify:
    runs-on: ubuntu-latest
    needs: [deploy-backend, deploy-frontend]

    steps:
      - name: Notify Slack
        uses: 8398a7/action-slack@v3
        with:
          status: ${{ job.status }}
          text: 'Deployment completed!'
          webhook_url: ${{ secrets.SLACK_WEBHOOK_URL }}
        if: always()
```

GitHub Projects Setup

1. Create Project Board

Board Structure:

- **Backlog**: All pending tasks
- **To Do**: Selected for current sprint
- **In Progress**: Currently being worked on
- **In Review**: PR created, awaiting review
- **Testing**: Deployed to staging, being tested
- **Done**: Completed and merged

2. Automation Rules

```
When issue is created → Add to Backlog
When PR is opened → Move to In Review
When PR is merged → Move to Done
When issue is closed → Move to Done
```

3. Labels

Priority:

- `priority: critical` (red)
- `priority: high` (orange)
- `priority: medium` (yellow)
- `priority: low` (green)

Type:

- `type: bug` (red)
- `type: feature` (blue)
- `type: enhancement` (purple)
- `type: documentation` (gray)
- `type: refactor` (pink)

Component:

- `component: backend` (blue)
- `component: frontend` (green)
- `component: ml` (yellow)
- `component: infrastructure` (orange)

Status:

- `status: blocked` (red)
- `status: in-progress` (yellow)
- `status: ready-for-review` (green)

Code Quality Tools Configuration

Backend: `.flake8`

```ini
[flake8]
max-line-length = 127
exclude =
    .git,
    __pycache__,
    venv,
    .venv,
    migrations,
    tests
ignore = E203, E266, W503
max-complexity = 10
```

Backend: `pyproject.toml`

```toml
[tool.black]
line-length = 127
target-version = ['py310']
include = '\.pyi?$'
exclude = '''
/(
    \.git
  | \.venv
  | migrations
)/
'''

[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = "test_*.py"
python_classes = "Test*"
python_functions = "test_*"
addopts = "-v --cov=app --cov-report=html --cov-report=term-missing"

[tool.mypy]
python_version = "3.10"
warn_return_any = true
warn_unused_configs = true
disallow_untyped_defs = true
```

Frontend: `.eslintrc.json`

```json
{
  "extends": [
    "react-app",
    "react-app/jest",
    "plugin:@typescript-eslint/recommended",
    "prettier"
  ],
  "rules": {
    "@typescript-eslint/explicit-function-return-type": "warn",
    "@typescript-eslint/no-unused-vars": ["error", { "argsIgnorePattern": "^_" }],
    "no-console": ["warn", { "allow": ["warn", "error"] }],
    "prefer-const": "error"
  }
}
```

Frontend: `.prettierrc.json`

```json
{
  "semi": true,
  "trailingComma": "es5",
  "singleQuote": true,
  "printWidth": 100,
  "tabWidth": 2,
  "useTabs": false,
  "arrowParens": "avoid"
}
```

Git Hooks with Husky

Install Husky

```bash
```

```
# In frontend directory
npm install --save-dev husky lint-staged
npx husky install
npm set-script prepare "husky install"

# In backend directory
pip install pre-commit
```

Frontend: `package.json`

```json
{
  "lint-staged": {
    "*.{ts,tsx}": [
      "eslint --fix",
      "prettier --write"
    ],
    "*.{json,md}": [
      "prettier --write"
    ]
  }
}
```

Backend: `.pre-commit-config.yaml`

```yaml
```

```yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.4.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-added-large-files
        args: ['--maxkb=1000']

  - repo: https://github.com/psf/black
    rev: 23.3.0
    hooks:
      - id: black

  - repo: https://github.com/PyCQA/flake8
    rev: 6.0.0
    hooks:
      - id: flake8

  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.3.0
    hooks:
      - id: mypy
        additional_dependencies: [types-requests]
```

Documentation Best Practices

README.md Structure

```markdown
```

Recruitment Matching Platform

Brief description of the project.

Features
- Feature 1
- Feature 2
- Feature 3

Tech Stack
- Backend: FastAPI, PostgreSQL
- Frontend: React, TypeScript
- ML: PyTorch, spaCy, OpenCV

Quick Start

Prerequisites
- Docker & Docker Compose
- Python 3.10+
- Node.js 18+

Installation
\```bash
git clone ...
cd recruitment-matching-platform
cp .env.example .env
docker-compose up
\```

Development
\```bash
Backend
cd backend
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --reload

Frontend
cd frontend
npm install
npm start
\```

Project Structure
[Link to detailed structure]

```
## Contributing
See [CONTRIBUTING.md](CONTRIBUTING.md)

## License
[License information]
```

CONTRIBUTING.md

```markdown
# Contributing Guidelines

## How to Contribute

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Write tests
5. Submit a pull request

## Development Setup
[Detailed setup instructions]

## Coding Standards
[Link to style guides]

## Testing Requirements
- Unit tests: >80% coverage
- All tests must pass
- No linting errors

## Pull Request Process
[Detailed PR process]

## Code of Conduct
[Link to code of conduct]
```

Monitoring & Alerts

GitHub Actions Notifications

Add to workflows:

```yaml
```

```yaml
- name: Notify on Failure
  if: failure()
  uses: 8398a7/action-slack@v3
  with:
    status: failure
    text: 'Build failed! Please check the logs.'
    webhook_url: ${{ secrets.SLACK_WEBHOOK_URL }}
```

Status Badges

Add to README.md:

```markdown
![Backend CI](https://github.com/your-org/recruitment-platform/workflows/Backend%20C
![Frontend CI](https://github.com/your-org/recruitment-platform/workflows/Frontend%2
[![codecov](https://codecov.io/gh/your-org/recruitment-platform/branch/main/graph/ba
```

Security Best Practices

1. **Never commit secrets:** Use GitHub Secrets
2. **Scan dependencies:** Use Dependabot
3. **Code scanning:** Enable GitHub Advanced Security
4. **Review permissions:** Limit access appropriately
5. **Sign commits:** Use GPG keys
6. **Audit logs:** Regularly review access logs
7. **Branch protection:** Enforce on main/develop

This setup provides a robust foundation for collaborative development with automated quality checks, clear processes, and comprehensive documentation.